



Agentic LLMs

From Retrieval to Tools to Autonomous Agents

01 RAG

02 Tool Calling

03 Agents

04 Skill

Contents

Three steps that turn a language model into an agent

01

RAG

Ground answers in outside knowledge

02

Tool Calling

Let the model act — call APIs & functions

03

Agents

Plan, act, observe, and loop toward a goal

01

Retrieval-Augmented Generation

Give the model the right knowledge, just in time.

Motivation

Anything after training simply isn't there.

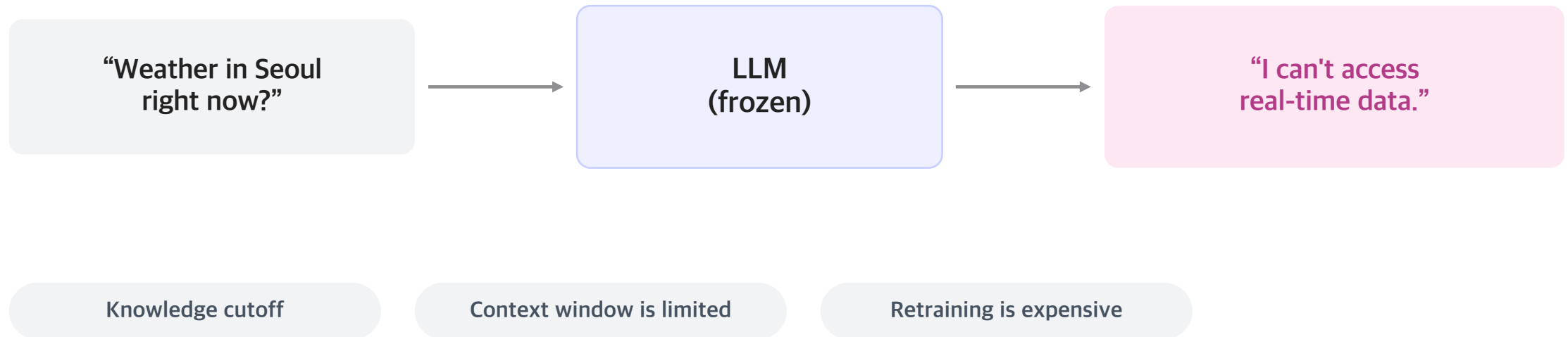
- The knowledge of LLM is limited to its pretraining and SFT data.
- Further training is computationally expensive.
- Incorporating new external knowledge into an LLM faces several challenges:
 - Limited context length,
 - Risk of introducing incorrect or noisy information, which can degrade performance.

👉 Therefore, a mechanism is needed to provide proper knowledge to the LLM based on the user's query.



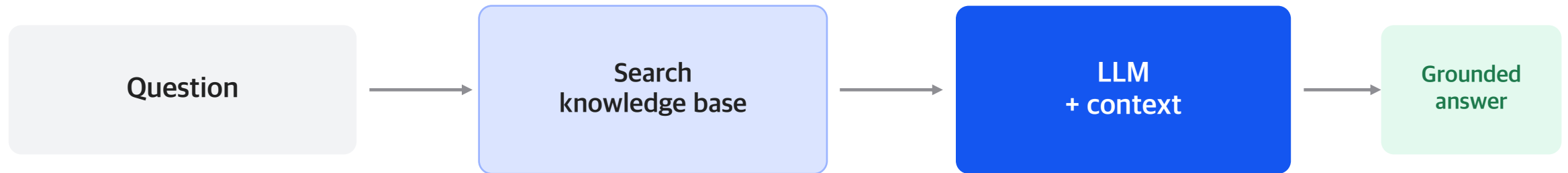
The problem: a model's knowledge is frozen

Anything after training simply isn't there.



The fix: look it up, then answer

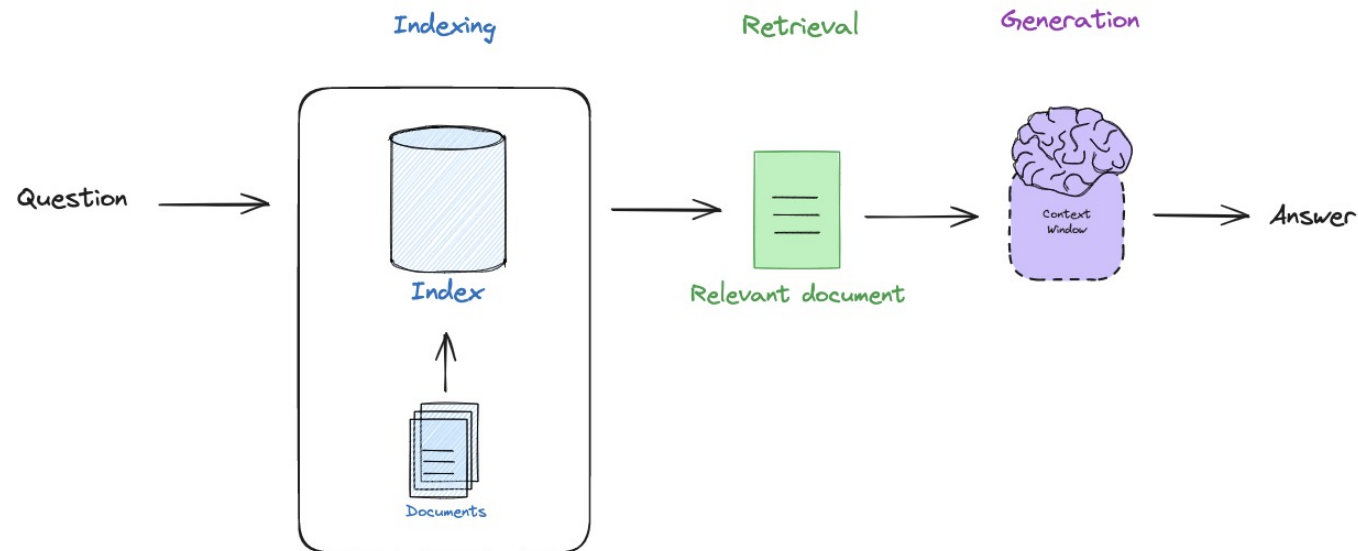
RAG hands the model fresh context before it replies.



Key idea → the model's weights never change — we just feed it better context at query time.

Retrieval-Augmented Generation (RAG)

Retrieve → Augment → Generate.



Retrieve

Retrieve relevant documents from a knowledge base

Augment

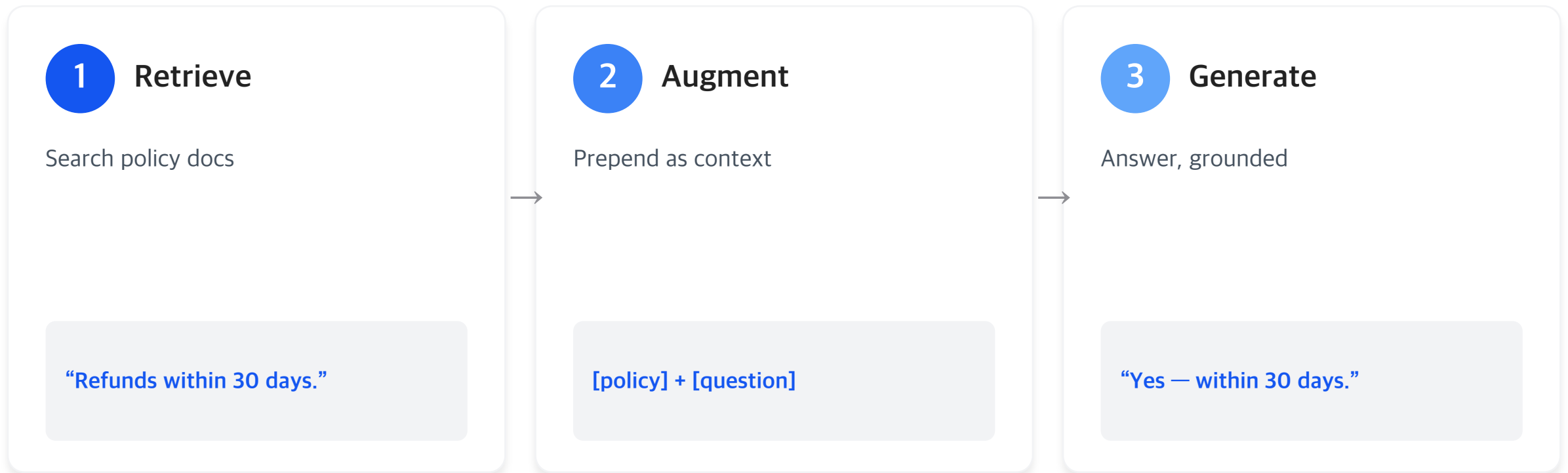
Add the retrieved information to the prompt along with the user's query

Generate

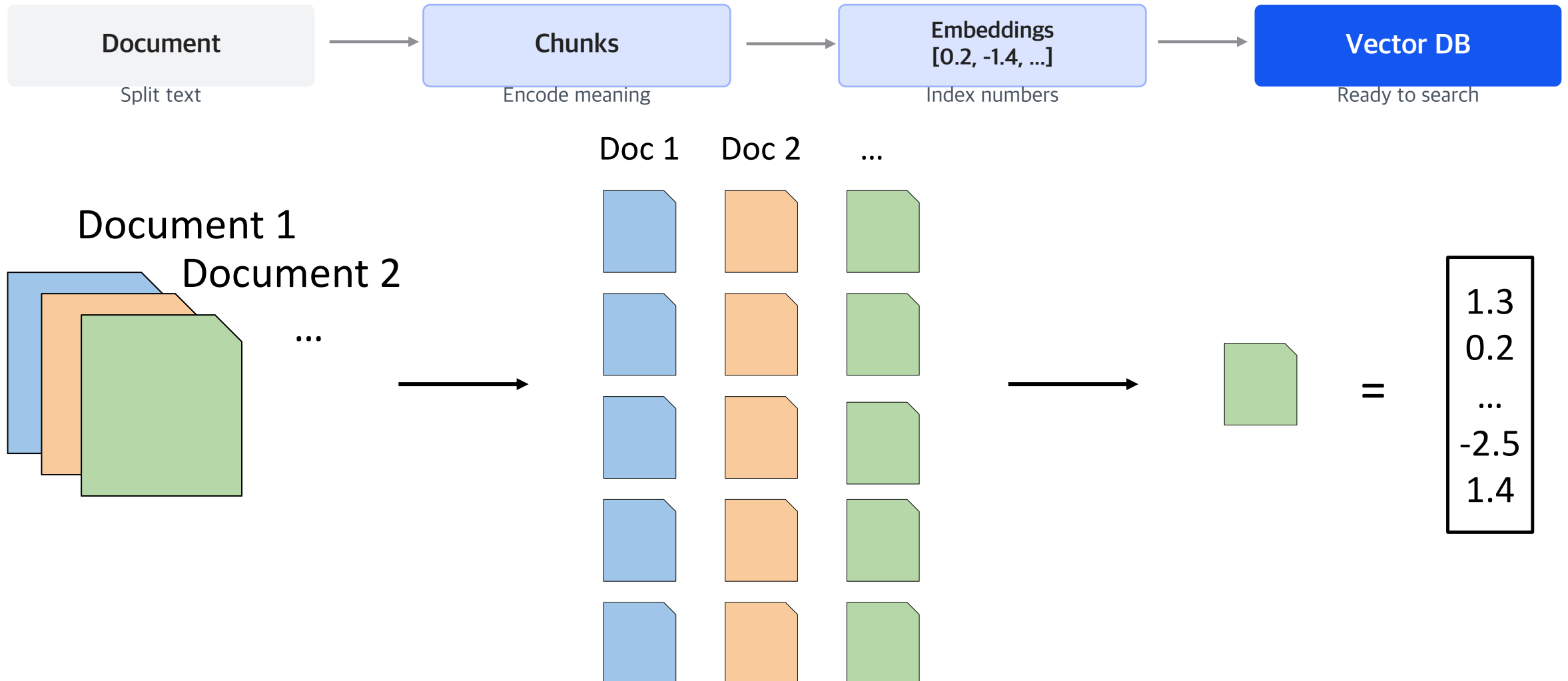
Generate a response based on the augmented context using the LLM

Worked example: a refund question

One query through all three stages.



Indexing: prepare the knowledge base before retrieving



Chunk size is a trade-off

How big should each piece be?

Too large

One chunk mixes several topics — retrieval pulls in noise the model must ignore.

e.g. a whole page in one vector

Too small

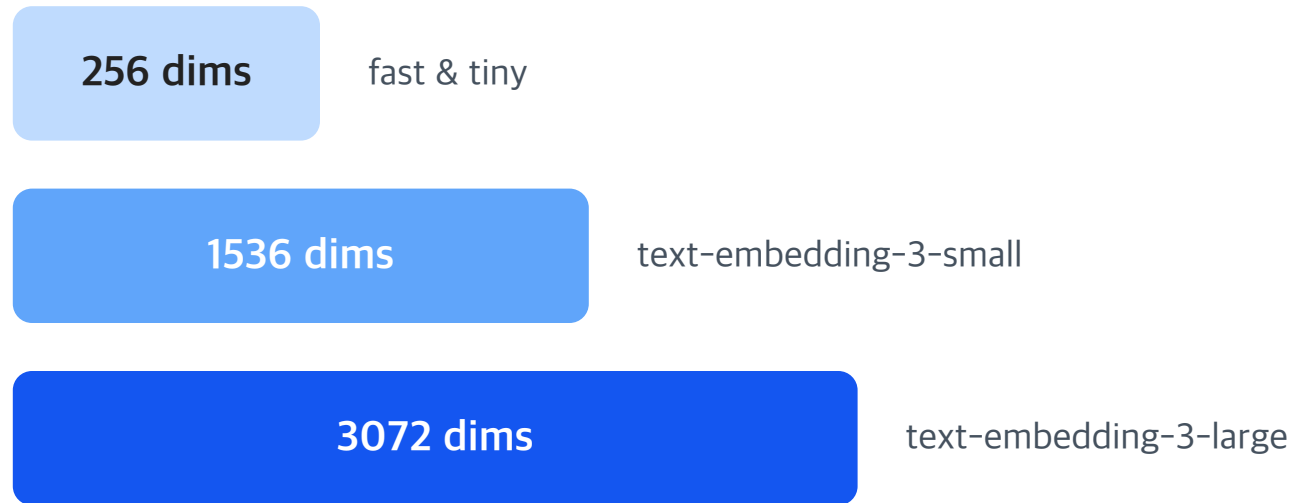
A sentence loses its surrounding context — the chunk no longer makes sense alone.

e.g. one line at a time

Sweet spot → about one paragraph (~500 tokens) with a little overlap between neighbors.

Embedding dimension: how rich is each vector?

More numbers per chunk = more nuance, but slower and bigger.



Rule of thumb → bigger isn't always better; pick the smallest dimension that keeps your accuracy.

Two ways to match text

By exact keywords, or by meaning.

Sparse · keyword match

BM25

“automobile” $\neq$ “car”

Matches the literal words. Great for names, codes, and rare terms — blind to synonyms.

Dense · meaning match

cosine similarity

“automobile” $\approx$ “car”

Embeddings place similar meanings nearby. Handles paraphrase — may miss an exact keyword.

BM25 vs Dense retrieval

Same question, two ways to rank — keyword overlap vs semantic meaning.

Query



“서울 날씨가 추워서 따뜻한 옷을 입는 게 좋을까?”

BM25 · keyword overlap

Top 3 results

1 서울 겨울철 평균 기온과 날씨 정보 8.7

서울의 겨울은 평균 -5~5°C로 **춥다**.

2 서울 날씨 예보 - 오늘과 내일 7.1

오늘 **서울**은 맑고, 내일은 눈이 내릴 예정.

3 한파주의보 발표 기준 5.3

기온이 -12°C 이하로 2일 이상 지속될 때...



Best for exact terms — names, IDs, codes. Blind to synonyms.

Dense · meaning similarity

Top 3 results

1 겨울철 옷차림 가이드: 어떻게 입을까? 0.92

날씨가 **추운** 날에는 보온성 좋은 **옷**을 여러 겹 입으세요.

2 기온이 낮을 때 건강 관리 방법 0.88

기온이 낮으면 체온 유지를 위해 **따뜻한** 옷을 입으세요.

3 겨울철 실내외 온도 차이와 주의 0.85

실외가 **추울** 때 **따뜻하게** 입고, 실내선 조절.



Captures intent — matches paraphrases. May miss a literal word.



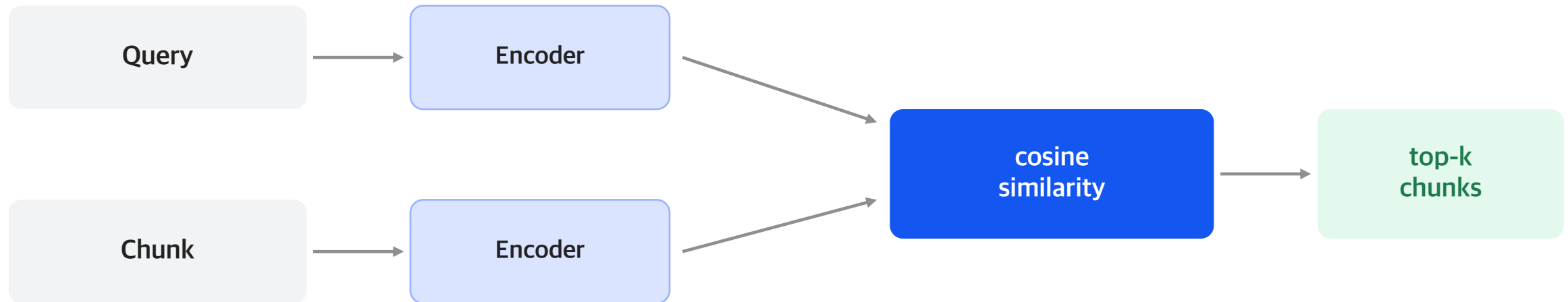
In one line: **BM25** = “how many of the same words overlap?” **Dense** = “how close in meaning?”

→ Production RAG often combines both (Hybrid Retrieval).

Retrieval

How to retrieve relevant documents for the query?

- Encode the query and document chunks **independently** using **bi-encoder**.
- Retrieve top-k most similar documents via cosine similarity among them.

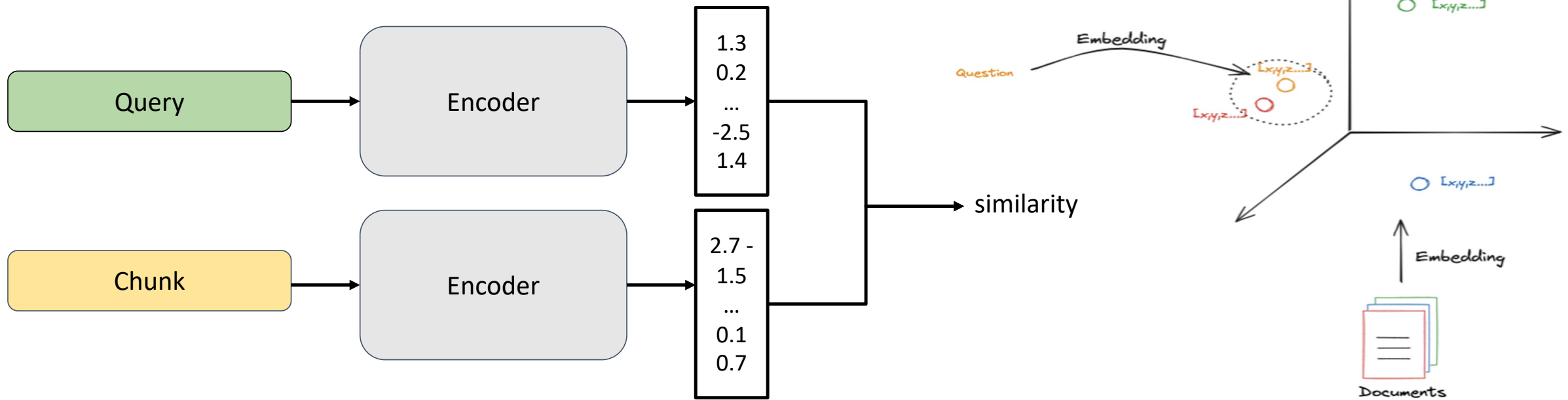


Chunk vectors are precomputed offline — at query time we only encode the question, so search stays fast.

Retrieval

How to retrieve relevant documents for the query?

- Encode the query and document chunks **independently** using **bi-encoder**.
- Retrieve top-k most similar documents via cosine similarity among them.



But the top result isn't always best

Separate encoding can rank a so-so chunk too high.

Bi-encoder top-3

- 1 Loosely related
- 2 Exactly answers it
- 3 Off-topic

The best chunk is buried at #2.

After reranking

- 1 Exactly answers it
- 2 Loosely related
- 3 Off-topic

A reranker pushes the true best to #1.

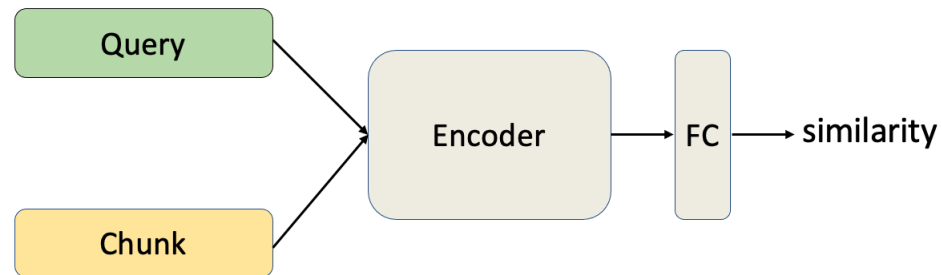
Reranker

Is semantic search enough?

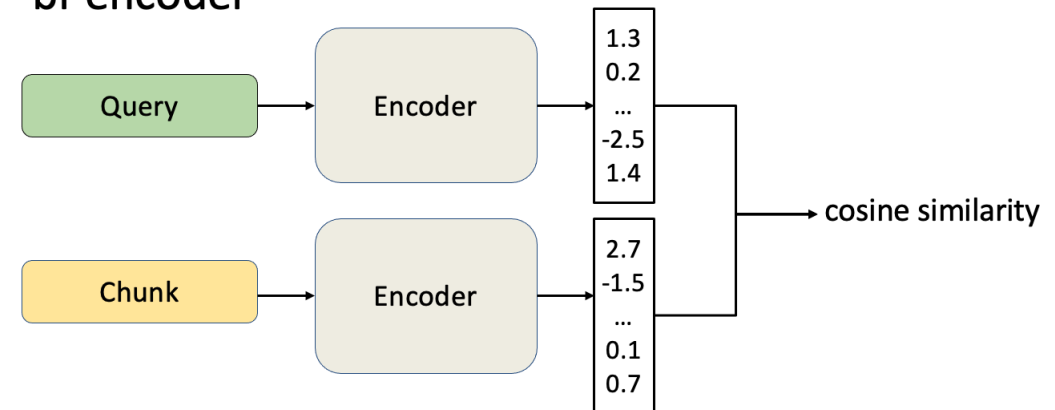
- Embedding similarity may not fully capture the true relevance between query and documents
- Bi-encoder is efficient but can be less precise

👉 Use a **cross-encoder** to compute more accurate relevance scores via cross-attention, and rerank the retrieved documents accordingly.

cross-encoder



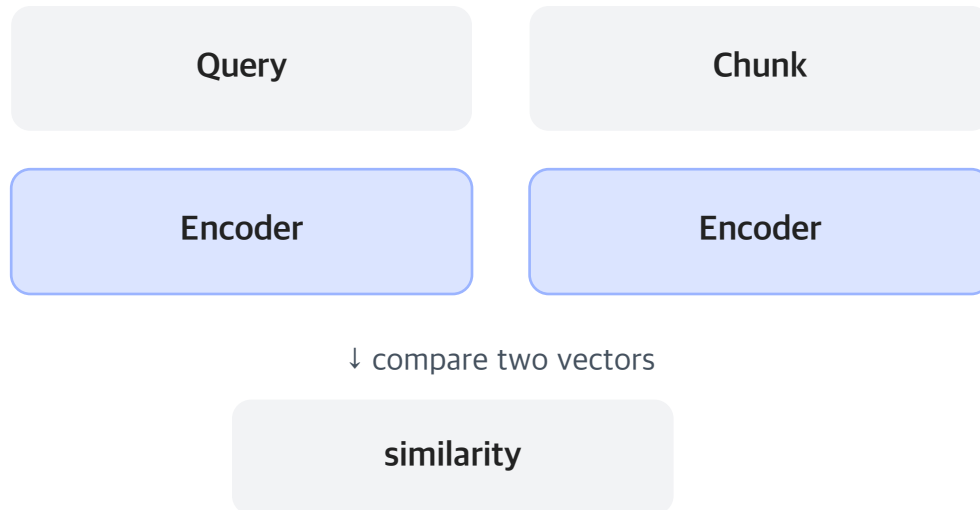
bi-encoder



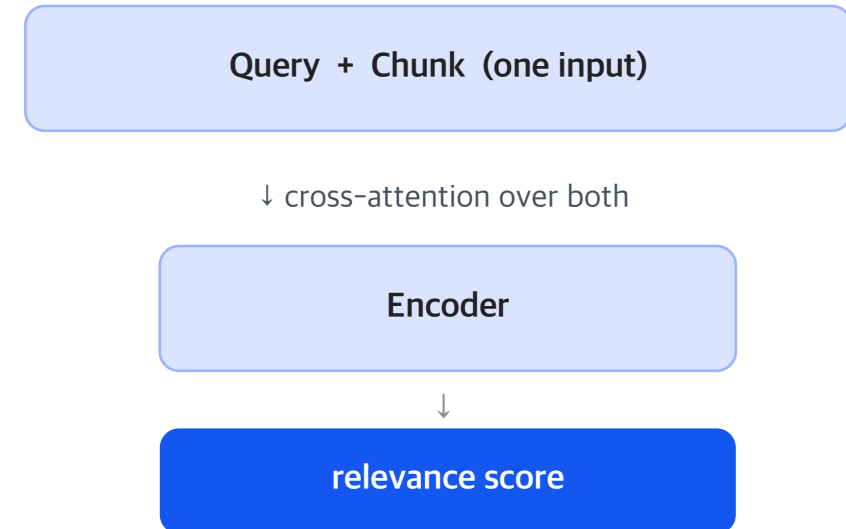
Reranking with a cross-encoder

Read query + chunk together for a sharper score.

Bi-encoder · fast

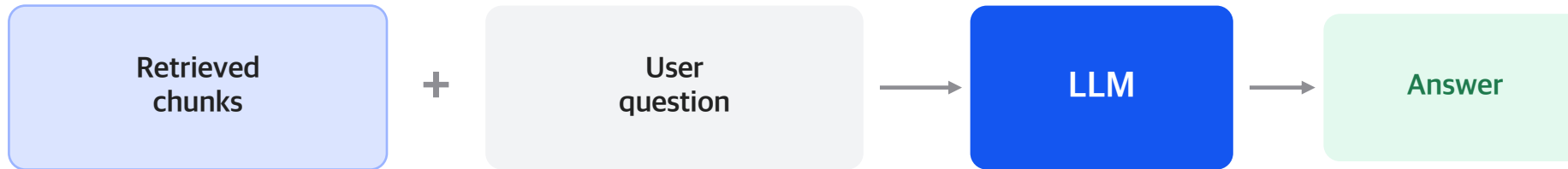


Cross-encoder · accurate



Augment & Generate

Drop the chunks into the prompt; the LLM answers from them.



Prompt template

```
Context: {retrieved_chunks}
Question: {user_question}
Answer:
```

```
from langchain.chains import RetrievalQA

# LLM
llm = ChatOpenAI(model="gpt-4o-mini")

# Prompt template
prompt = ChatPromptTemplate.from_template("""
You are a helpful assistant.

Context:
{context}

Question:
{question}

Answer:
""")

# RAG chain
qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever, # vectorstore.as_retriever()
    chain_type_kwargs={"prompt": prompt}
)

# 실행
qa_chain.invoke({"query": "What is BM25?"})
```

Did retrieval work? Four metrics

Each answers a different question about the ranked results.

Precision@k

Of the top-k, how many are relevant?

accuracy

Recall@k

Of all relevant docs, how many made the top-k?

coverage

NDCG@k

Are the best docs ranked near the top?

ranking

MRR

How soon does the first good doc appear?

first hit

When plain RAG falls short

Top-k chunks are independent — they don't connect the dots.

“How are Alice and the Berlin project related?”

Alice joined in 2021

Berlin project launched

...led by the new hire

✗ No edge links them — the model can't reason across hops.

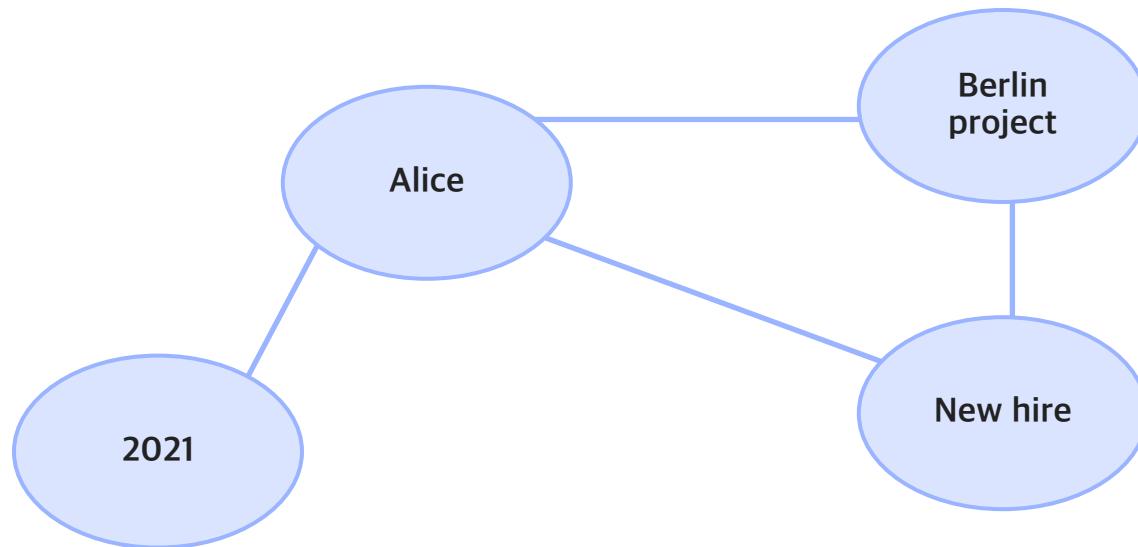
Disconnected evidence

Multi-hop reasoning is hard

No global structure

Graph RAG: connect the evidence

Store knowledge as a graph, then retrieve connected paths.

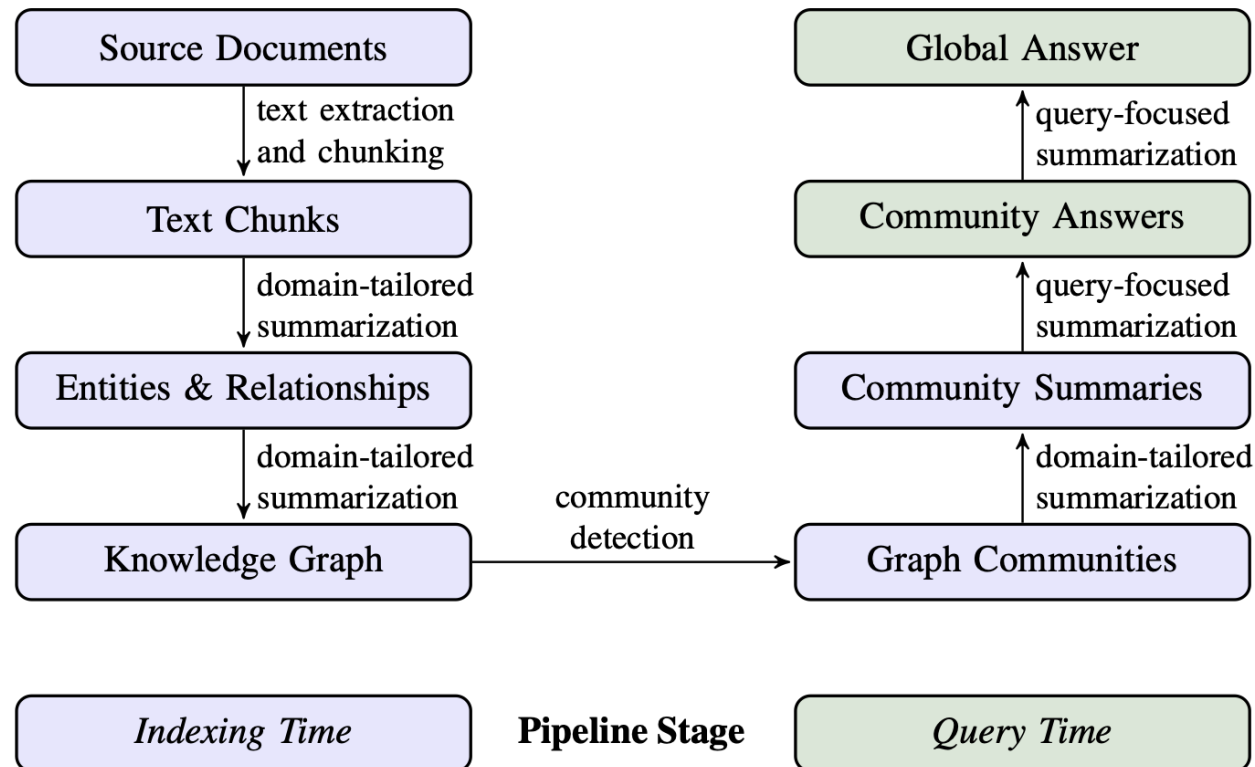


Why a graph?

Entities become **nodes**, relationships become **edges**. Retrieval can now follow connections and answer **multi-hop** questions a flat list can't.

Graph RAG pipeline

Build the graph offline; answer by community at query time.



Two phases, side by side

Heavy lifting happens once, offline.

Indexing time

- 1 Extract** entities, relations, claims from text
- 2 Build** connect them into a knowledge graph
- 3 Cluster** group into communities + summarize

Query time

- 1 Map** each community drafts a partial answer
- 2 Parallel** all communities run independently
- 3 Reduce** aggregate into one global answer

02

Tool Calling

Let the model act on the world, not just talk about it.

RAG reads. It can't act.

Some questions need a live action, not a stored document.

RAG can answer

“What is our refund policy?”

“Summarize this contract.”

RAG cannot

“What's the weather in Seoul now?”

“Add this row to my spreadsheet.”

Need → a way to run real-time data fetches, computations, and actions.

Tool calling, in one line

The model decides which function to run, with what arguments.



The model doesn't run code itself — it outputs a structured request, your app executes it, and the result goes back in.

Example: a weather tool

Step 1 — write a normal Python function.

```
@tool
def get_weather(location: str) -> str:
    """Get the current weather in a location."""
    url = "https://api.openweathermap.org/..."
    params = {"q": location, "appid": API_KEY}

    resp = requests.get(url, params=params)
    data = resp.json()
    return f"It's {data[...]}°C in {location}."
```

WHAT THE LLM NEEDS

Just three things

name

get_weather

description

what it does

parameters

location: str

Step 2 — describe it as JSON

The function is turned into a schema the model can read.

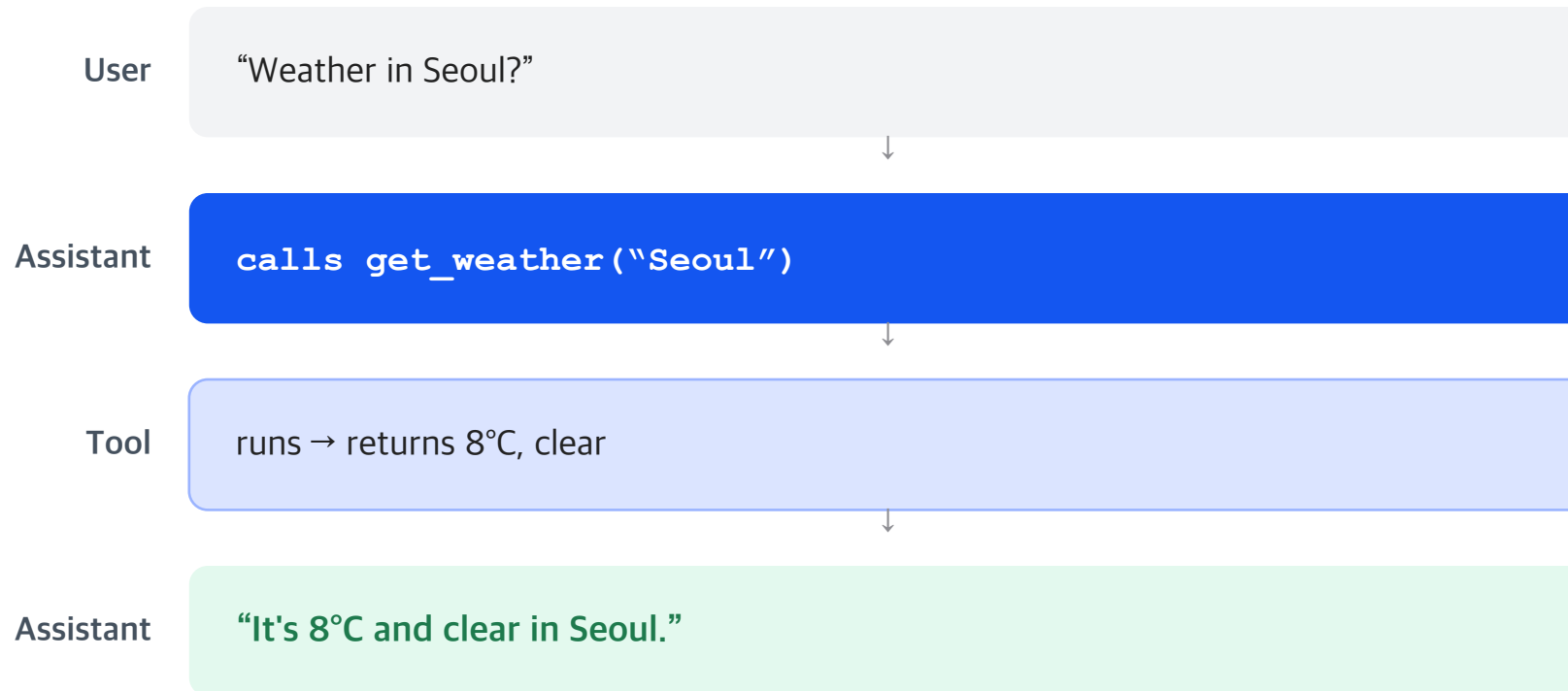
```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "Get current weather",
    "parameters": {
      "location": {"type": "string"},
      "required": ["location"]
    }
  }
}
```

Why JSON?

The model never sees your code — only this **contract**. The **description** tells it **when** to call; **parameters** tell it **how**.

Step 3 — the call loop

Five messages turn a question into a grounded answer.



How does the model learn to call tools?

Two training signals — often used together.

SFT · learn from examples

labeled tool-use demos → imitate

Teaches the **correct format** and stable, common behaviors.
Great warm-up, but limited to what the demos covered.

RL · learn from reward

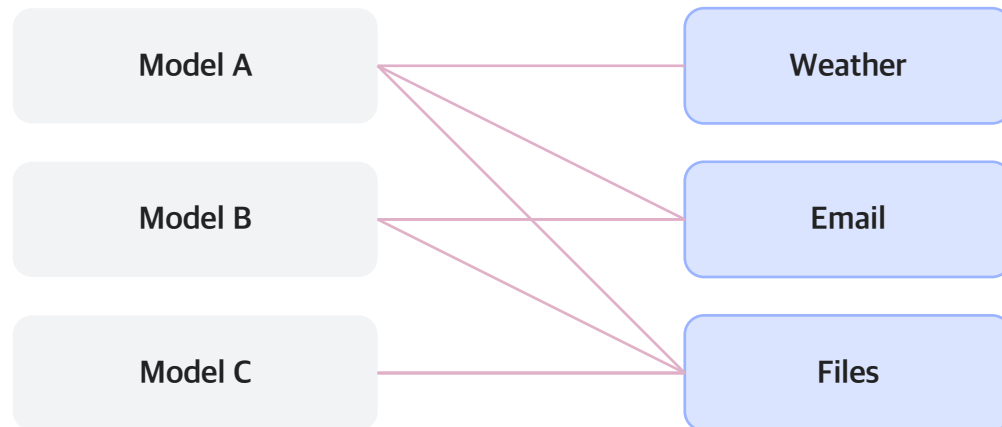
try → get reward → improve

Better for **multi-step reasoning** and complex tool chains.
Common recipe: **SFT to warm up, then RL to optimize.**

The M×N problem

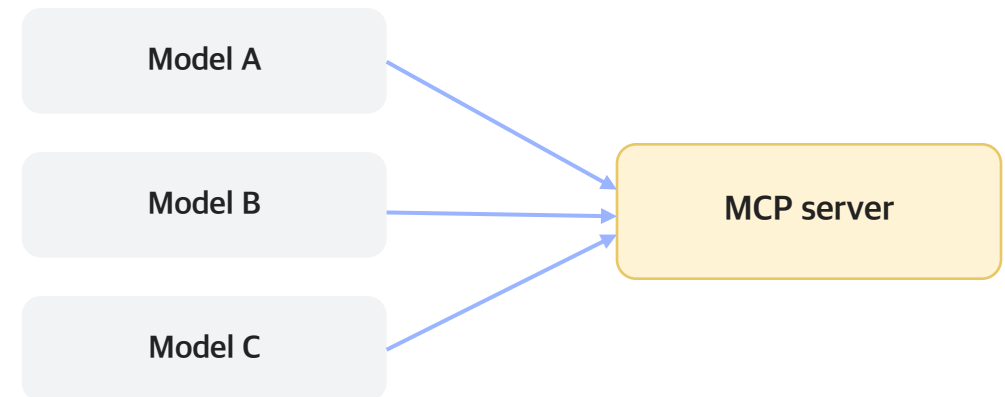
Every model wiring up to every tool by hand doesn't scale.

Without a standard



$M \times N$ custom integrations

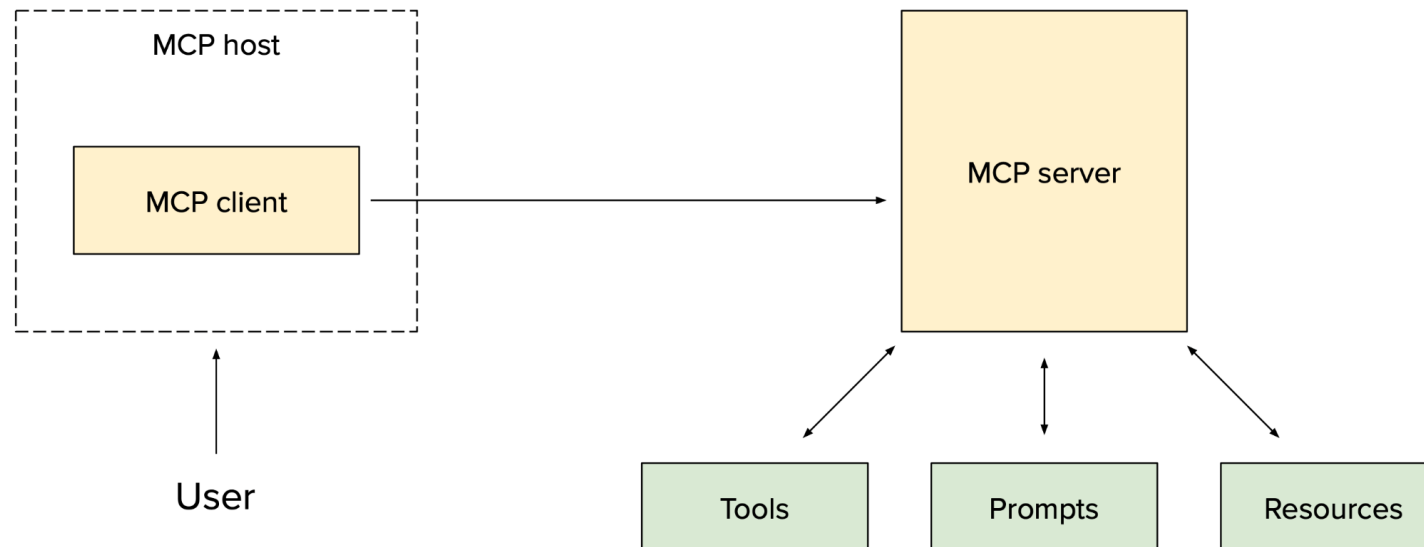
With MCP



one protocol, plug in once

Model Context Protocol (MCP)

A standard interface to tools, prompts, and data.



Tools — functions to run

Prompts — reusable templates

Resources — data to read

MCP in practice

Connect Claude Code to Notion — its tools appear instantly.

```
notion-search [read-only] [open-world]
notion

Tool name: notion-search
Full name: mcp__notion__notion-search

Description:
Perform a search over:
- "internal": Semantic search over Notion workspace and connected sources (Slack, Google Drive, Github, Jira, Microsoft Teams, Sharepoint, OneDrive, Linear). Supports filtering by creation date and creator.
- "user": Search for users by name or email.

Auto-selects AI search (with connected sources) or workspace search (workspace-only, faster) based on user's access to Notion AI. Use content_search_mode to override.
Use "fetch" tool for full page/database contents after getting search results. Each result's "url" field contains a page ID for Notion results (pass directly to fetch tool's "id" param) or a full URL for external connector results (Slack, Google Drive, etc.). Set page_size (default 10, max 25) and max_highlight_length (default 200, 0 to omit) as low as possible to minimize response size.
To search within a database: First fetch the database to get the data source URL (collection://...) from <data-source url="..."> tags, then use that as data_source_url. For multi-source databases, match by view ID (?v=...) in URL or search all sources separately.
Don't combine database URL/ID with collection:// prefix for data_source_url. Don't use database URL as page_url.
<example description="Search with date range filter (only documents created in 2024)">
{
  "query": "quarterly revenue report",
  "query_type": "internal",
  "filters": {
```

What just happened

- 1 Add one MCP server
- 2 Notion's tools auto-register
- 3 Model can search & edit pages

No custom glue code.

03

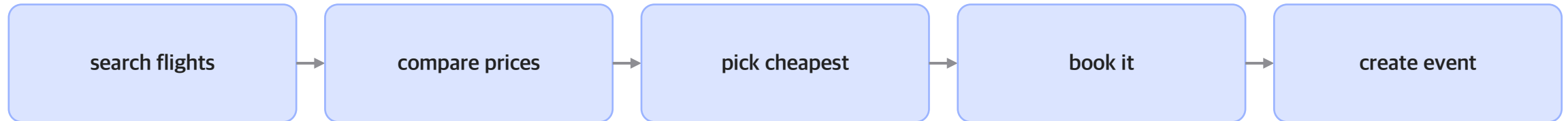
Agents

Plan, act, observe, and loop toward a goal.

One tool call is rarely enough

Real tasks need several steps, decided on the fly.

“Book the cheapest flight to Tokyo next Friday and add it to my calendar.”



A single call can't do this — each step depends on the result of the last. We need something that loops, decides, and adapts.

What is an agent?

An LLM that runs in a loop until the goal is met.

An **agent** is a system that uses an LLM to **autonomously plan, act, and iterate** toward a goal — calling tools, reading the results, and deciding the next step on its own.

Plans

breaks a goal into steps

Acts

calls tools to make progress

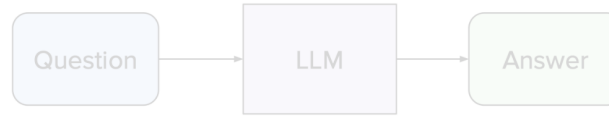
Iterates

uses feedback to adapt

From answer machine to agent

Each step adds a new ability on top of the last.

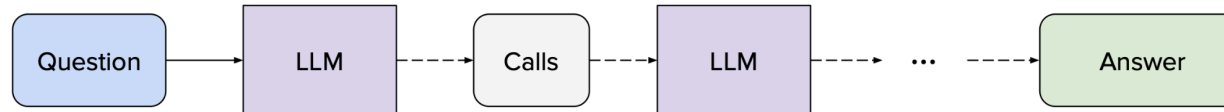
Traditional



Reasoning



Agent



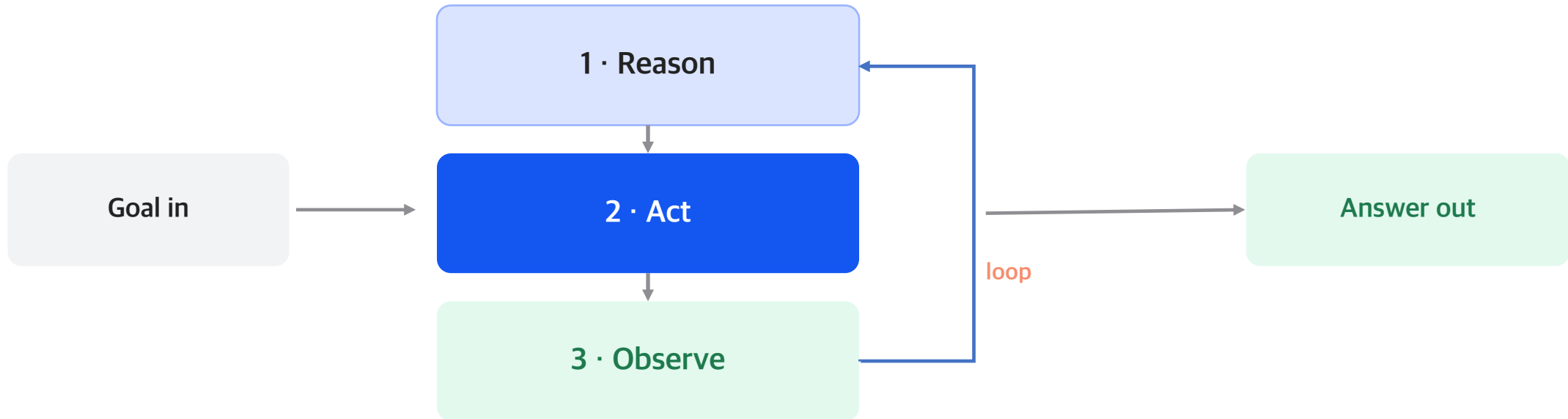
Traditional $Q \rightarrow A$, in one shot

+ Reasoning thinks before answering

Agent calls tools in a loop

ReAct = Reason + Act

The loop at the heart of most agents.



Reasoning picks the next action; observing the result feeds the next thought — repeat until done.

ReAct in action

A weather agent reasons and acts, step by step.

“Do I need an umbrella in Seoul today?”

Reason	Thought	I should check today's forecast for Seoul.
Act	Act	<code>get_weather("Seoul")</code>
Observe	Obs	rain, 80% chance
Reason	Thought	80% is high — an umbrella is worth it.
Finish	Answer	“Yes, bring an umbrella today.”

When one agent isn't enough

Big tasks split better across specialists.

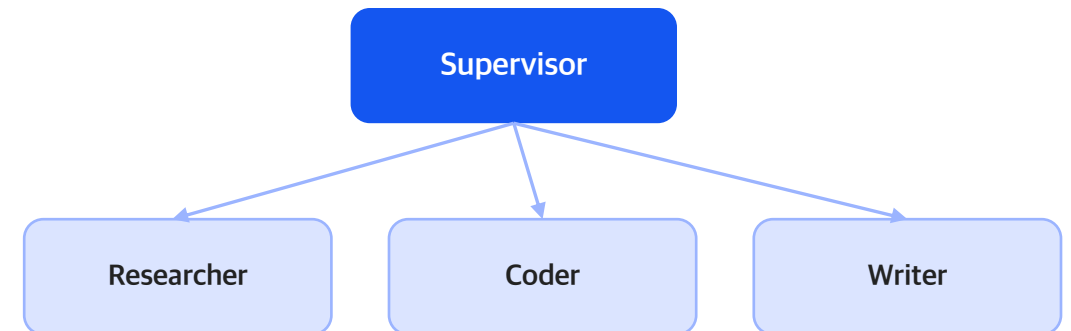
Single agent



one agent
does it all

Juggling research, coding, and writing in one context gets tangled and error-prone.

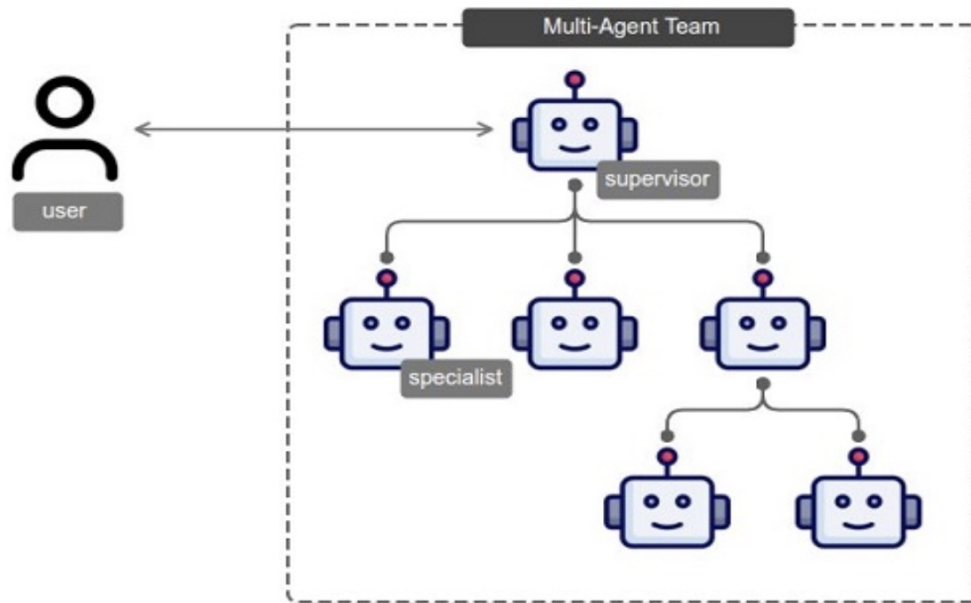
Multi-agent



Each specialist owns one job; a supervisor coordinates and combines.

A multi-agent team

A supervisor delegates to specialists, who can delegate further.



How it works

User states the goal

Supervisor plans & routes work

Specialists each handle a sub-task

Combine results merge into one answer

The full stack, assembled

Each layer unlocks the next.

1

RAG

gives the model knowledge

Reads

2

Tool Calling

lets the model act

Acts

3

Agents

let it plan & loop

Decides

Agentic LLM = a reasoning model that knows (RAG), acts (tools), and decides (agent loop).

04

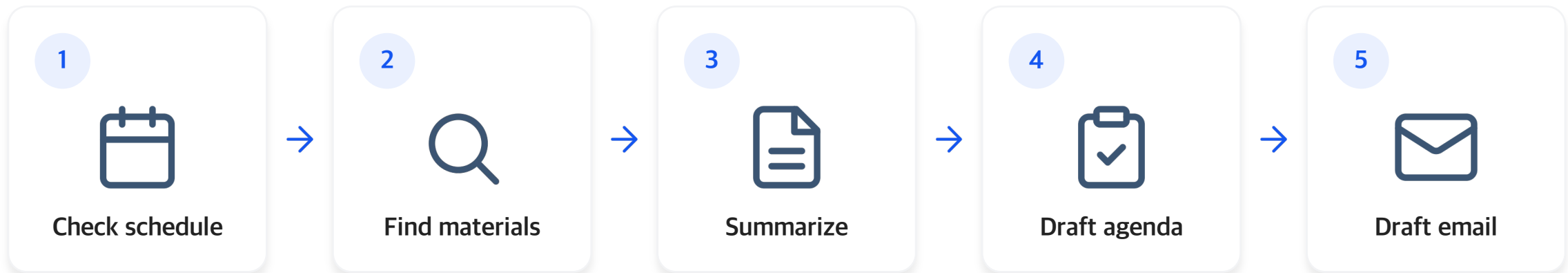
Skills

Reusable workflows that package know-how

Motivation

Tool calling is powerful — but most real requests are multi-step workflows.

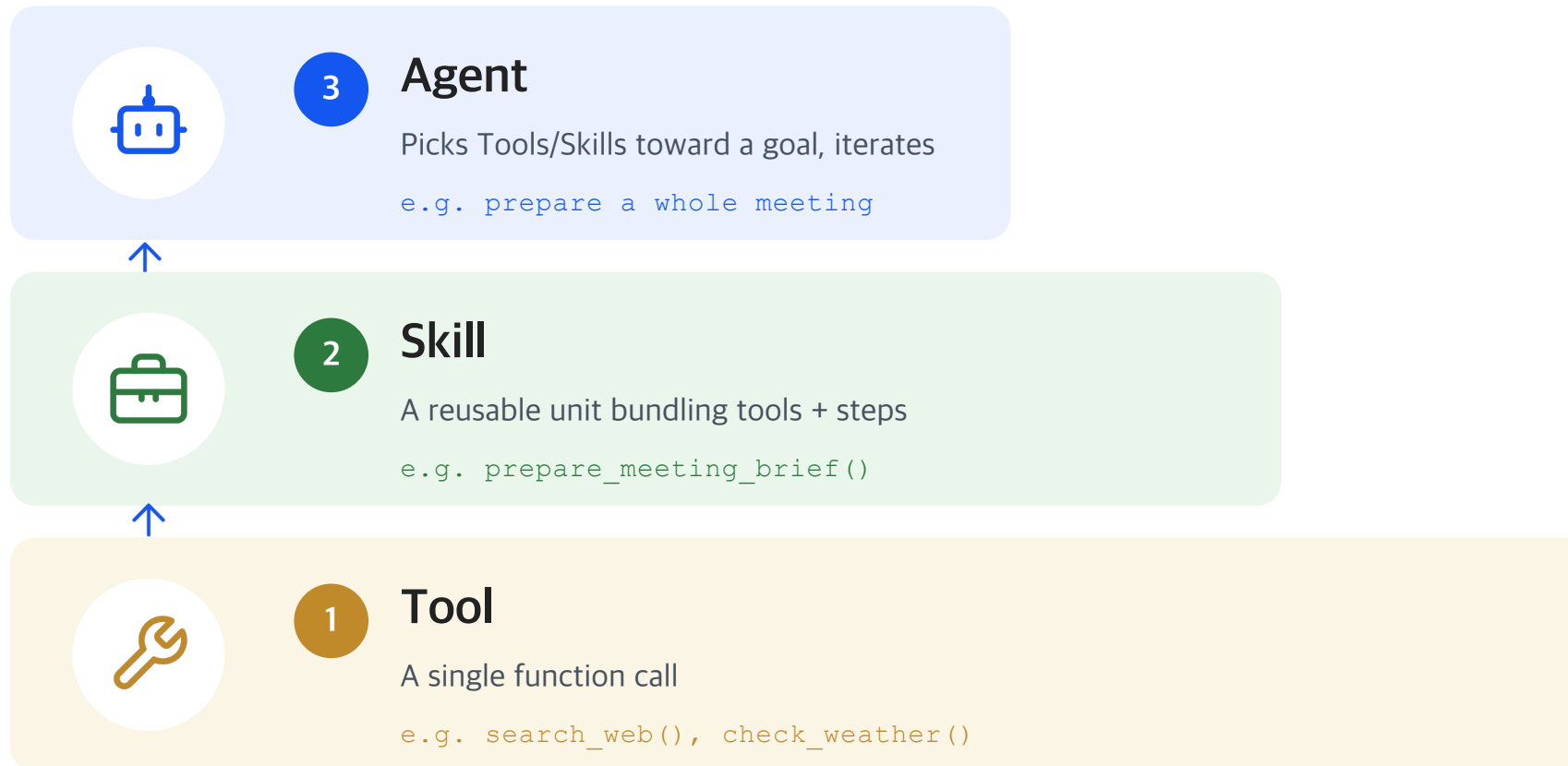
Example request: "Prepare this week's research meeting"



Re-running this chain from scratch every time is inefficient — **a reusable workflow** is the answer.

Tool → Skill → Agent

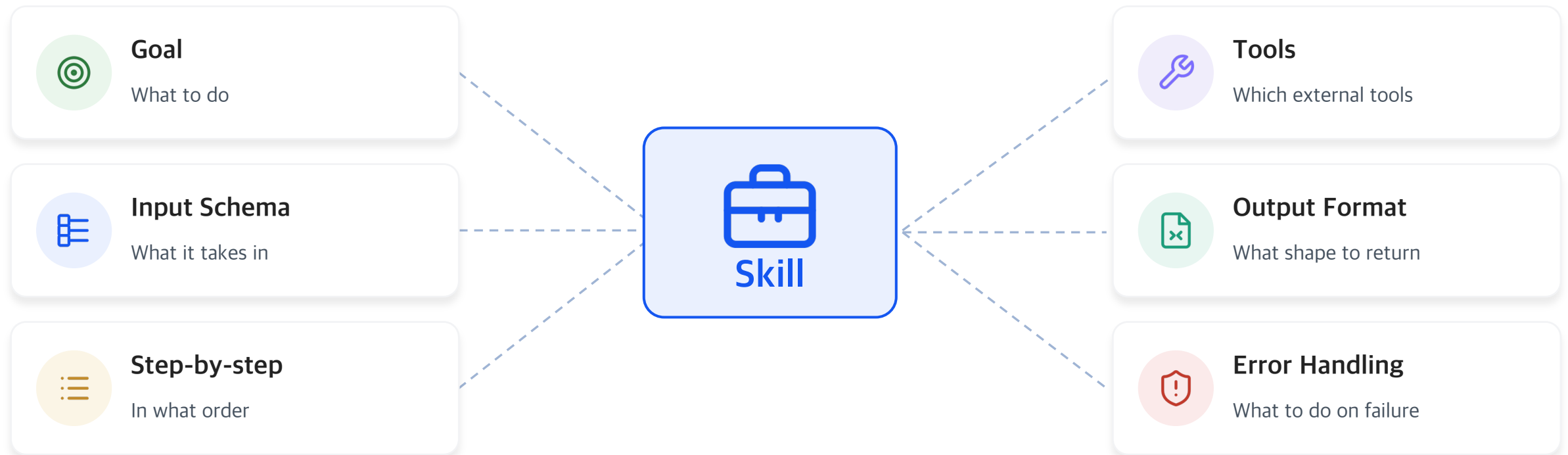
Three layers that build on each other and work together.



- ### Key Takeaways
- 1 Tools do one thing.
 - 2 Skills package know-how.
 - 3 Agents orchestrate both.

What is inside a Skill?

A Skill defines goal, input, steps, tools, output, and error handling — systematically.



A Skill isn't just a tool name — it's a **task package with the know-how built in.**

Tool Calling vs. Skills

Same building blocks, different scope of work.

	Tool Calling	Skills
 Unit	A single function / API call	A reusable mini-workflow
 Role	Fetch info or do one action	Chain multiple steps & tools
 Example	<pre>check_weather('Seoul'), search_paper('RAG')</pre>	<pre>prepare_meeting_brief(topic), make_teaching_slide(topic)</pre>
 Strength	Simple and flexible	Automates repeats, consistent
 Limit	Re-plans the order each time	Needs upfront design & upkeep

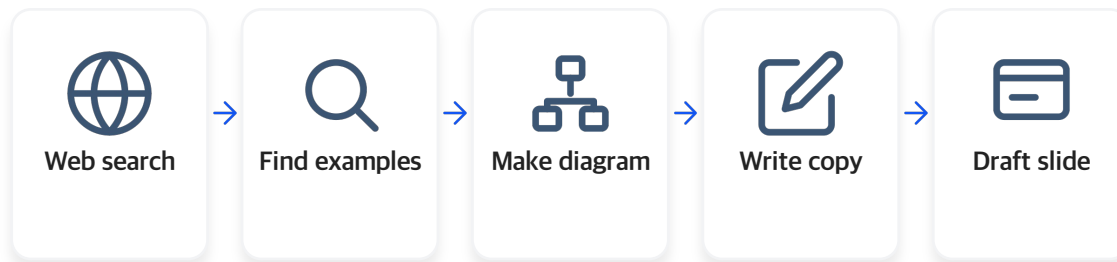
Analogy: Tool = one screwdriver / Skill = the assembly manual + the tools it needs

Easy Example

One request, two ways to fulfill it.

Request: "Make a 1-slide explainer on BM25 vs Dense Retrieval"

Only Tool Calling



Re-decides the order every time

Using a Skill

 `make_teaching_slide(topic, audience)`

Internally: search + examples + diagram + slide draft

A Skill compresses many tools and steps into **one reusable unit of work.**

When are Skills useful?

Four signals that a workflow deserves to become a Skill.

1



Repetitive tasks

Weekly reports, meeting prep

2



Multi-tool tasks

Search + compute + write

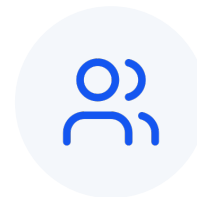
3



Consistency matters

Standard emails, JSON, slide drafts

4



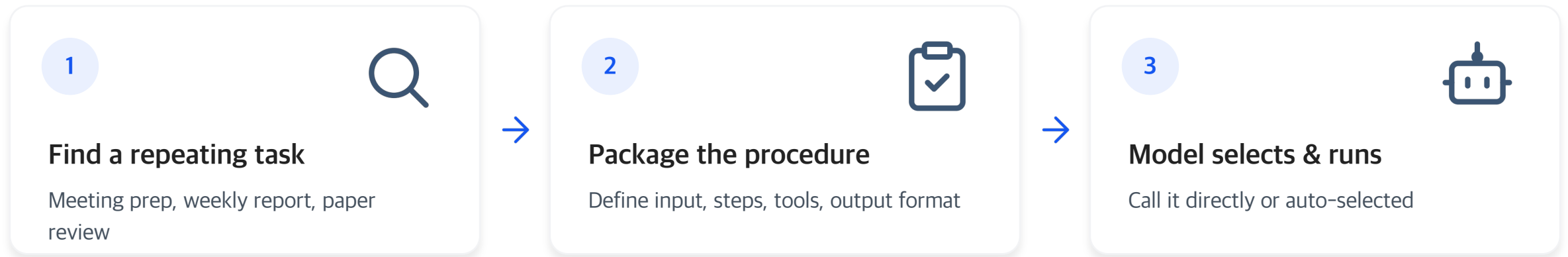
Team-wide reuse

Similar quality for anyone

If the same chain repeats often, **make it a Skill** instead of wiring tools each time.

How to use Skills

Find the repeat, package the procedure, let the model run it.



Example Skill



Name: `write_paper_review`

Input: paper PDF, review form

Uses: `extract_text`, `summarize`, `checklist`

Output: structured review draft

The point: store the **good way of doing it** in a reusable form.

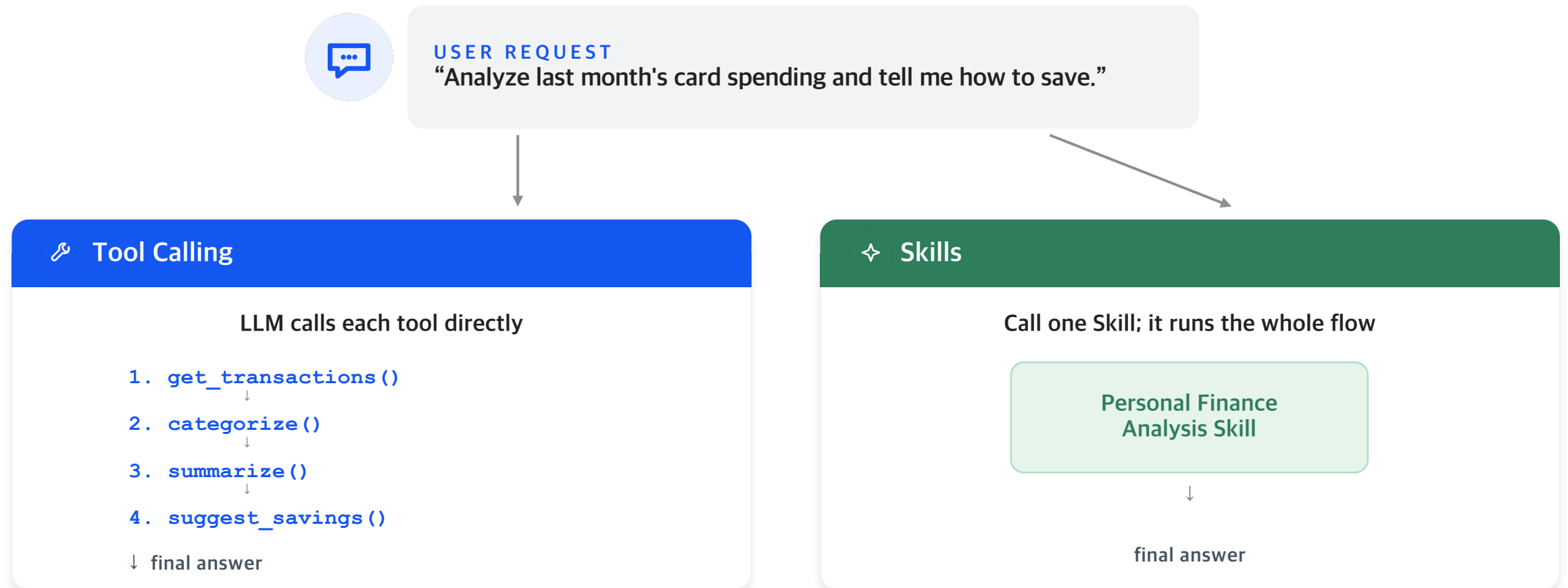


Appendix (The usage of skills)

Retrieval · Tools · Agents — the building blocks of agentic LLMs.

The difference at a glance

Same request, two ways to get the job done.



Difference ① — a difference of role

A tool is an ability; a Skill is a method.

🔧 Tool Calling = “ability to use a tool”



Like using a **hammer**
to drive in a nail.

- Single-function focused
- LLM decides which tool each time
- You build params & parse results

✦ Skills = “way to do the work”



Like knowing the **procedure**
to build a finished chair.

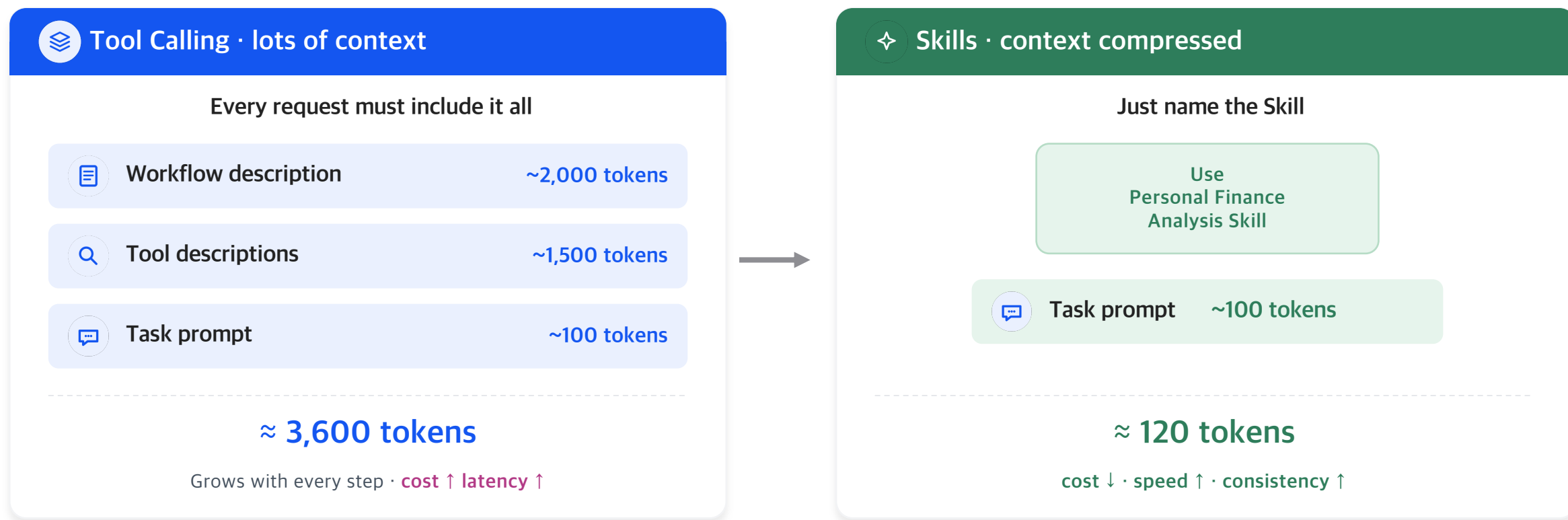
- Bundles many steps into one workflow
- Uses several tools internally on its own
- User calls just one Skill



In short: a Tool is a **Function**; a Skill is a **Workflow**.

Difference ② — context usage

Skills compress what the model must carry each time.



30x less context for the same task — the saving compounds in long agent runs.

A real scenario

Reviewing whether a paper fits a conference — a multi-tool job.



USER REQUEST

“Can you review this NIPS paper? I've attached the PDF.”



paper.pdf

This task needs several tools, in order:



Read PDF
analyze paper



Web Search
NIPS info



Paper Search
related work



Compare
novelty check



Find Risks
rejection risk



Write Report
final review



Not one tool call — a composite task: several tools combined in sequence.

Doing it with Tool Calling

The LLM picks each tool and orders them itself — every time.

LLM composes the tools manually

```
1 read_pdf("paper.pdf")
2 web_search("NIPS 2026 CFP")
3 paper_search("recent NIPS data-mining work")
4 web_search("NIPS accepted trends 2024-2025")
5 analyze_and_compare()
6 write_review_report()
```

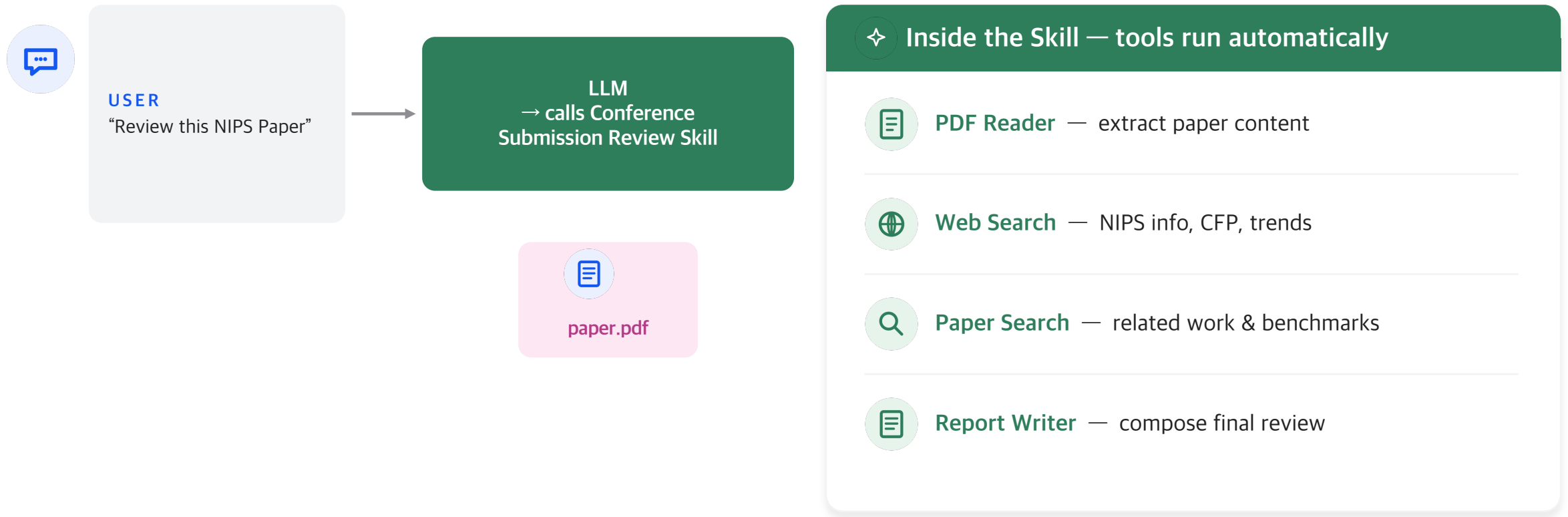
Problems

- Must inject a long instruction (criteria, format, cautions) every time
- Output quality varies request to request
- High context cost — the whole workflow must be described each run

→ Flexible, but inefficient and inconsistent.

Packaging it as a Skill

Define once; afterwards the LLM just calls the Skill by name.



The workflow and tool usage live **inside** the Skill — the LLM just follows it, reliably.

What the Skill file looks like

skill.md — goal, tools, workflow, and output format in one place.

```
skills/conference_submission_review/skill.md
```

Conference Submission Review Skill

Goal

Decide cold-bloodedly if the paper fits the venue.

Tools to Use

PDF Reader · Web Search · Paper Search · Report Writer

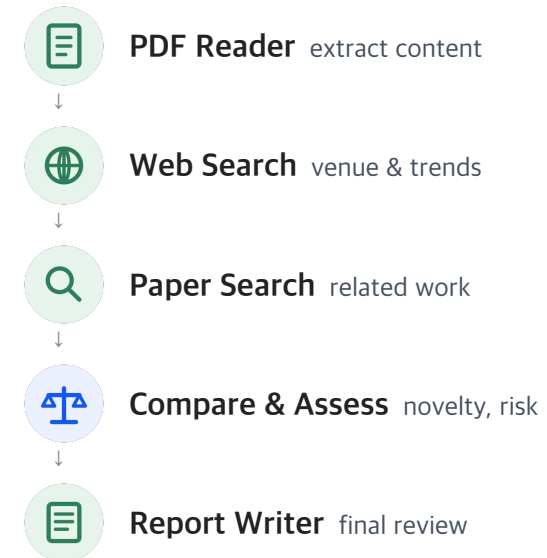
Workflow

1. Read the PDF, grasp the core claims
2. Research the target venue & recent trends
3. Find ≥3 related papers, compare novelty
4. Assess rejection risk (venue fit, rigor)
5. Suggest fixes with a score

Output Format

Verdict · Summary · Strengths · Risks · Score · Fixes

Workflow preview



What the Skill file looks like

skill.md — goal, tools, workflow, and output format in one place.

```
skills/conference_submission_review/skill.md
```

Conference Submission Review Skill

Goal

Decide cold-bloodedly if the paper fits the venue.

Tools to Use

PDF Reader · Web Search · Paper Search · Report Writer

Workflow

1. Read the PDF, grasp the core claims
2. Research the target venue & recent trends
3. Find ≥ 3 related papers, compare novelty
4. Assess rejection risk (venue fit, rigor)
5. Suggest fixes with a score

Output Format

Verdict · Summary · Strengths · Risks · Score · Fixes

Output

“The paper is interesting and seems suitable for ICDM. The experiments are broad and useful.”

Expected score: **4.0**



Problems

- Too positive
- Rejection risk underrated
- No venue-fit analysis

Update Skill description as you want!

skill.md — goal, tools, workflow, and output format in one place.

skills/conference_submission_review/skill.md

Conference Submission Review Skill

Goal

Decide cold-bloodedly if the paper fits the venue.

Tools to Use

PDF Reader · Web Search · Paper Search · Report Writer

Workflow

1. Read the PDF, grasp the core claims
2. Research the target venue & recent trends
3. Find ≥3 related papers, compare novelty
4. Assess rejection risk (venue fit, rigor)
5. Suggest fixes with a score

Critical Review Rules

- Always assume the reviewer is skeptical.
- Do not say "interesting" unless the novelty is clearly explained.
- If the paper claims practical relevance, check whether the deployment scenario is realistic.
- Always include at least three possible reviewer objections.

Output Format

Verdict · Summary · Strengths · Risks · Score · Fixes

After · improved version

“May fit the Applied Track, but the current version has a **borderline risk**: reviewers may see it as application-specific rather than a general contribution.”

Expected score: **3.5 / 5**



Improvements

- ✓ Cold, honest assessment
- ✓ Venue fit + related work compared
- ✓ Risks flagged early, with fixes

Tool Calling vs Skills — recap

Six lenses, then the practical takeaway.

	 Tool Calling	 Skills
Concept	LLM picks & combines each tool	Tools + steps in a Skill; call by name
Tool use	Use any tool flexibly	Fixed tools in a fixed order
Consistency	Quality varies per request	Stable, repeatable output
Context	Whole workflow injected each time	Just the Skill name
Maintenance	Hard to tweak prompts	Edit skill.md → instant update
Trade-off	Instant, very flexible	Efficient, stable, scalable



In practice: prototype fast with Tool Calling → harden repeated, complex jobs into Skills (hybrid).



Thank you

Retrieval · Tools · Agents — the building blocks of agentic LLMs.